Vidarbha Youth Welfare Society's

Prof. Ram Meghe Institute of Technology & Research,

Badnera-Amravati (M.S) 444701.

Laboratory Manual

Semester IV

Subject: - Data Structure and Algorithm Lab (4SX08)

Department of Computer Science and Engineering(IOT)

1. Vision and Mission of Institute and Department

Vision of the Institute

To become a pace-setting Centre of excellence believing in three universal values namely Synergy, Trust and Passion, with zeal to serve the Nation in the global scenario.

Mission of the Institute

M1: To achieve the highest standard technical education through state of the art pedagogy and enhanced industry institute linkage.
M2: To inculcate the culture of research in core and emerging areas.
M3: To strive for overall development of studies so as to nurture ingenious technocrats as well as responsible citizens.

Vision and Mission of Department of Computer Science and Engineering(IOT)

Vision of the Department

To achieve excellence in the field of Computer Science & Engineering and Internet of Things through dedication & collaboration, so as to meet global standards in education, research and industrial applications and for nurturing young professionals & entrepreneurs with strong belief in social & ethical values for progress of the society.

Mission of the Department

M1: To provide state of the art facilities in order to achieve excellence in teaching – learning, innovative activities, entrepreneurship and research.
M2: To frame curricula and syllabi based on technological needs of industry along with due consideration for moral, ethical and social values, for nation building.
M3: To encourage inter-disciplinary projects and research through industry- institute linkages so as to bridge the gap between industry and academia.
M4: To develop technology-based solutions for relevant societal problems such as environment, health, etc. and impart experiential learning through inter disciplinary/ collaborative research.

2.Program educational objective (PEO's), program outcomes (PO's) and Program Specific Outcomes (PSO's)

- **Program educational objective (PEO's)**

PEO1: Provide students with opportunities to acquire the ability to apply the fundamentals of mathematics, sciences and engineering to quantitatively analyze engineering problems.

PEO2: Provide students with opportunities to develop industry oriented innovative design skills, including the ability to formulate problems, to think creatively, to synthesize information, and to communicate effectively.

PEO3: Provide students with opportunities to develop the ability to use modern experimental techniques; collect, analyze, and interpret experimental data; and effectively communicate the results.

PEO4: Provide opportunities to prepare students with the diverse skills needed to be successful and competent engineers and/ or to pursue higher studies.

- **Program Outcomes (PO's)**

Engineering Graduate will be able to:

PO1. Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2. Problem analysis: Identify, formulate, review research literature, and analyse complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3. Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4. Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5. Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.

PO6. The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7. Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO8. Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9. Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
PO10. Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
PO11. Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
PO12. Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

- **Program Specific Outcomes (PSO's)**

PSO1: Develop different problem solving skills in order to solve industrial problems based on principles and concepts of CSE & IoT.

PSO2: Comprehend various application development tools and their implementation for IoT applications.

PSO3: To design and implement CSE and IoT based solution for technology based industrial problems

3. Syllabus, Course outcomes and Mapping of CO's with PO's

Course Title	:	**Data Structure Lab(4SX08)**
Faculty	:	Prof. V. M. Sarad Prof. A. R. Solanke
Course Type	:	Laboratory(4SX08)
Compulsory / Elective	:	Compulsory
Teaching Method	:	Laboratory: 02 Hrs / week (04 Batches)
Course Credits	:	01 Credit for Laboratory
Course Assessment (Practical)	:	1. Laboratory: Lab Performance & Final Submission by Internal + Viva-Voce Semester End External Examination Practical: 50 % Internal (25 Marks) + 50 % External (25 Marks).
Grading Policy	:	2. Grading Table

Grades	Marks
A+	10
A	9
B+	8
B	7

C+	6
C	5
D	4

	Syllabus 4SX03 (Unit wise Course Contents) **Subject: Data Structure and Algorithm**	
Unit-1	Introduction to Algorithms and Data Structures: Introduction, Algorithms, Asymptotic, Notation, Introduction to Data Structures, Types of Data Structures, Data Structure, Operations.	08
Unit-2	Arrays: Introduction, Types of Arrays, Representation of One-Dimensional Array in Memory, Array Traversal, Insertion and Deletion, Sorting and Searching, Representation of Multi-Dimensional Array in Memory. Linked List: Introduction, Linked Lists – Basic Concept, Types of Linked List.	08
Unit-3	Stacks: Introduction to Stacks, Stack Operations. Queues: Introduction, Queues - Basic Concept, Queue Operations, Basics of Circular Queues and Priority Queues.	06
Unit-4	Trees: Introduction, Basic Concept, Binary Tree, Binary Tree Representation, Binary Tree Traversal, Binary Search Tree,	06
Unit-5	Graphs: Introduction, Basic Concept, Graph Terminology, Graph Implementation, Shortest Path Algorithm, and Graph Traversal.	06
Unit-6	Sorting and Searching: Introduction, Sorting Techniques: Selection Sort, Insertion Sort, Bubble Sort, Quick Sort, Merge Sort. Searching Techniques: Linear Search, Binary Search.	08
	Total	42

Text Books:

1. Seymour Lipschutz, "Data Structures with C", Schaum's Outline Series.

Reference Books:

1. E. Balagurusamy, "Data Structures using C", McGraw Hill Education, 2018
2. Forouzan, Gilberg, "Data Structures and Algorithms",
3. CENGAGE Learning.
4. Reema Thareja, "Data Structures using C", Oxford University Press, 2011.
5. Arpita Gopal, "Magnifying Data structures", PHI (EEE), 2010.
6. Ellis Horowitz and Sartaj Sahani, "Fundamentals of Data Structures", CBS Publications

Course Learning Objectives:

1. Familiarize with basic algorithms and their efficiency.
2. Implement basic operations on arrays, stacks, linked list, Queue.
3. Compare sorting and searching techniques.

Course Outcomes:

After Successful completion of this course, the student will be able to:

Sr. No.	Course Outcome
4SX08.1	Use his/her understanding/knowledge of Data Structure and Algorithm to design /conduct experimentation related to the given aim/objective.
4SX08.2	Students will use stacks, Queue, array and link list in problem-solving.
4SX08.3	Interpret results, formulate findings, draw suitable conclusion and respond to inquiries, related to given experiment.

- **Mapping of Course Outcomes with Program Outcomes**

Course	PO1	PO2	PO3	PO4	PO10
4SX08.1	**1**	**-**	**1**	**2**	-
4SX08.2	**-**	**1**	**1**	**-**	-
4SX08.3	**-**	**2**	**1**	**1**	1

Correlation levels: 1. Low 2. Medium 3.High

- **Mapping of Course Outcomes with Program Specific Outcomes**

Course	PSO 1	PSO 2	PSO 3
4SX08.1	**-**	**-**	**-**
4SX08.2	**-**	**1**	**1**
4SX08.3	**-**	**-**	**2**

Correlation levels: 1. Low 2. Medium 3.High

4. <u>Gradation Policy for Laboratory Assessment of each Practical for B.Tech Programme</u>

The practicals are of three types –

1. That involves lab equipment such as oscilloscopes, component development systems, signal generators (Test bench)
2. That involves Computers (Simulation)
3. That involves Field work (Community Engagement Project/ Field Project (CEP/FP), mini project, capstone projects etc.)

Following are the components involved in the gradation of the practicals except Community Engagement Project/ Field Project (CEP/FP), mini project, capstone projects etc. by the respective teachers

P1: Attempt and/or attention for execution of Practical task.
P2: Competency (Skill/ability) for performing particular task.
P3: Performance and timely submission of Practical records
P4: Ability to analyze results and draw conclusions.
P5: Question and Answers / Viva-Voce.

Grade	P1	P2	P3	P4	P5
A+	Performance in First attempt; exceptional attention to task	Able to apply skill effectively	Excellent performance & submission of practical record along with results and conclusion by consecutive practical day.	Highly insightful analysis with accurate, logical, and well-developed conclusions	Appropriate and accurate answers
A	Performance in First attempt; excellent attention to task	Good skill application	Effective performance & submission of practical record along with results and conclusion by consecutive practical day.	Clear, accurate analysis with well-reasoned conclusions	Relevant and well-explained answers
B+	Performance in First attempt; above average attention to task	Satisfactory skill application	Good performance & submission of practical record one week late then timely date of submission	Generally correct analysis with mostly appropriate conclusions	Mostly appropriate answers
B	Performance in second attempt; average attention to task	Minimal understanding skill	Acceptable performance & submission of practical record one	Minimal analysis with partially correct or	Partially appropriate answers

		application	week late then timely date of submission	incomplete conclusions	
C+	Performance in second attempt; below average attention to task	Somewhat skill application	Inconsistent performance & submission of practical record two week late then timely date of submission	Limited analysis with conclusions lacking clarity or accuracy.	Limited but somewhat relevant answers
C	Performance in third attempt or more; somewhat attentive	Severe difficulty in applying skills	Poor performance & submission of practical record two week late then timely date of submission	Minimal analysis with weak or unsupported conclusions	Weak or incomplete answers
D	Performance in third attempt or more; least attentive	Able to apply skills with help from faculty	Very poor performance & very late submission of practical record	Unable to analyze results; conclusions incorrect or missing	Incorrect or irrelevant answers

Mapping of Practicals gradation with marks

Grades	Marks
A+	10
A	9
B+	8
B	7
C+	6
C	5
D	4

5. PRACTICALS

Department of Computer Science & Engineering (IOT)

Subject: Data Structure and Algorithm Lab **Semester IV** **Code: 4SX08**

Sr. No	LIST OF PRACTICALS
1	To study and execute the Linear search method.
2	Understand the Linear data Structure and execute the Insertion, Deletion operation on it.
3	To study and execute the Bubble Sort method.
4	To study and execute the Binary Search method.
5	Define the stack structure and execute the push, pop operation on it.
6	Summarize the Queue structure and execute the insertion, deletion operation on it.
7	Discuss the Insertion sorting operation and implement on given DATA.
8	Implement Selection sort for a given list of integers and display results after every pass also analyze time and space complexity for the same.

Practical No-01

Aim: - To study and execute the Linear search method.

Objective: Implementation of searching operations for a Key value in a given list of integers and analyze the complexities for the same.

Linear search
DATA – {11, 22, 33, 44, 55, 66, 77, 88, 99, 12}
KEY- {66}

Software Required: pycharm-community-2024-1.4

Theory:

a) Linear search:-

Linear search technique is also known as sequential search technique. Linear search is a method of searching an element in a list in sequence. In this method, the array is searched for the required element from the beginning of the list/array or from the last element to first element of array and continues until the item is found or the entire list/array has been searched.

Algorithm: (Linear Search) A linear array DATA with N elements and a specific ITEM of information are given. This algorithm finds the location LOC of ITEM in the array DATA or sets LOC: =0.

1. [Initialize] Set K: =1 and LOC: =0.
2. Repeat Steps 3 and 4 while LOC=0 and K<=N.
3. If ITEM=DATA [K], then: Set LOC: =K.
4. Set K: =K+1. [Increments counter.] [End of step 2 loop.]
5. [Successful.]

If LOC=0, then:
Write: ITEM is not in the array
DATA Else:
Write: LOC is the location of
ITEM. [End of If Structure.]

6. Exit.

Time complexity: Worst Case Analysis:

In the worst case analysis, we calculate the upper bound on running time of an algorithm. We must know the case that causes the maximum number of operations to be executed. For Linear Search, the worst case happens when the element to be searched (x in the above code) is not present in the array. When x is not present, the search () function compares it with all the elements of arr[] one by one. Therefore, the worst case time complexity of linear search would be O(n) .

Average Case Analysis:

In average case analysis, we take all possible inputs and calculate computing time for all of the

inputs. Sum all the calculated values and divide the sum by the total number of inputs. We must know (or predict) the distribution of cases. For the linear search problem, let us assume that all cases are uniformly distributed (including the case of x not being present in the array). So we sum all the cases and divide the sum by n. Following is the value of average case time complexity.

$$
\begin{aligned}
C(n) &= 1*1/n+2*1/n+3*1/n+\ldots\ldots+n*1/n \\
&=(1+2+3+\ldots+n)*1/n \\
&=n*(n+1)/2 *1/n \\
&=(n+1)/2 \\
&=O(n)
\end{aligned}
$$

Best Case Analysis:

In the best case analysis, we calculate a lower bound on the running time of an algorithm. We must know the case that causes the minimum number of operations to be executed. In the linear search problem, the best case occurs when x is present at the first location. The number of operations in the best case is constant (not dependent on n). So time complexity in the best case would be $O(1)$.

Program:

Result & Discussion:

Conclusion

Questions:

1. What is data structure?
2. What are the types of data structure?
3. What is meant by algorithm?

Practical No-02

Aim: - Understand the Linear data Structure and execute the Insertion, Deletion operation on it.

Objective: Implementation of Linear data Structure to carry out various operations on array like Insertion, Deletion with any unsorted array.

DATA={10,54,23,1,4,5,7,3,44,66}

Software Required: pycharm-community-2024-1.4

Theory:

Array:

An array data structure or simply an array is a data structure consisting of a collection of elements (values or variables), each identified by at least one array index or key. An array is stored so that the position of each element can be computed from its index tuple by a mathematical formula. Array types are often implemented by array structures; however, in some languages they may be implemented by hash tables, linked lists, search trees, or other data structures.

One-dimensional arrays

A one-dimensional array (or single dimension array) is a type of linear array. Accessing its elements involves a single subscript which can either represent a row or column index. As an example consider the C declaration int an Array Name[10];

Multidimensional arrays

For a two-dimensional array, the element with indices i,j would have address $B + c \cdot i + d \cdot j$, where the coefficients c and d are the row and column address increments, respectively.

More generally, in a k-dimensional array, the address of an element with indices i1, i2, …, ik is

$B + c1 \cdot i1 + c2 \cdot i2 + \ldots + ck \cdot ik$.

For example: int a[3][2];

1. **Algorithm:-**

INSERT (LA, N, K, ITEM)

Here LA is a linear array with N elements and K is a positive integer such that K<=N. This algorithm inserts an element ITEM into the Kth position in LA.

1. [Initialize Counter.] Set J: =N.
2. Repeat Steps 3 and 4 while J>=K.
3. [Move J^{th} element downward.] Set LA [J+1]:= LA [J].
4. [Decrease counter.] Set J: =J-1.

   [End of Step 2 loop.]
5. [Insert element.] Set LA [K]:=ITEM.

6. [Reset N.] Set N: =N+1.
7. Exit.

2. DELETE ( ):

DELETE (LA, N, K, ITEM)

Here LA is a linear array with N elements and K is a positive integer such that K<=n. This algorithm deletes the Kth elements from LA.

1. Set ITEM: =LA [K].
2. Repeat for J=k to N-1:
   [Move J + 1st element upward.] Set LA [J]:=LA [J+1].
   [End of loop.]
3. [Reset the number N of elements in LA.] Set N: =N-1.

Exit.

Pseudo code:

Program:

Result and Discussion:

Conclusion:

Questions: -

1. What is an array?
2. Give the limitations of array.
3. Differentiate between 1-dimentional and 2-dimensional array.
4. Differentiate between static and dynamic memory allocation.

Practical No-03

Aim: To study and execute the Bubble Sort method.

Objective: Implement bubble sort for a given list of integers and display results after every pass give the time and space complexity for the same.

DATA – {98, 90, 12, 34, 20, 150, 54, 31, 86, 53}

Software Required: pycharm-community-2024-1.4

Theory:

Bubble Sort:-

Bubble sorting is a simple sorting technique in the sorting algorithm. In a bubble sorting algorithm, arrange the elements of the list by forming pairs of adjacent elements. It means repeatedly step through the list which we want to sort, compare two items at a time and swap them if they are not in the right order. Another way to visualize the bubble sort algorithm is as its name, the smaller element bubble to the top.

Steps:-

- Traverse a collection of elements:-
- Move from the front to the end
- Bubble" the largest value to the end using pairwise comparisons and swapping
- If we have N elements…And if each time we bubble an element, we place it in its correct location…Then we repeat the "bubble up" process N – 1 times.
- After passing 1 first largest element kept at last and the same other passes.

1. Complexity:-

1. Best Case performance – When the list is already sorted, we require only one pass to check, hence O(n).
2. Worst Case performance – When the list is in the reverse order, n number of comparisons and swap are required to be done n number of times, hence O(n2).
3. Average Case performance – O(n2)
4. It does not require any extra space for sorting, hence O(1) extra space.

2. Algorithms:-

BUBBLE SORT (DATA, N)

Here DATA is an array with N elements. This algorithm sorts the elements in DATA.

1. Repeat Steps 2 and 3 for K=1 to N-1.
2. Set PTR: =1. [Initializes pass pointer PTR]
3. Repeat while PTR<=N-K: [Executes pass.]
   (a) If DATA [PTR]>DATA [PTR+1], then:
   Interchanges DATA [PTR] and DATA [PTR+1]
   [End of If structure.]

(b) Set PTR:=PTR+

[End of inner loop.]

[End Step1 of outer loop.]

4. Exit.

Program:

Result and Discussion:

Conclusion:

Questions:

1. What is an ordered list?
2. What is a bubble sort and how do you perform it?
3. How many passes are required to sort an array?
4. What is time and space complexity?

Practical No-04

Aim: To study and execute the Binary Search method.

Objective: Implement the searching in a given list of integers with Key value and analyze the complexities for the same.

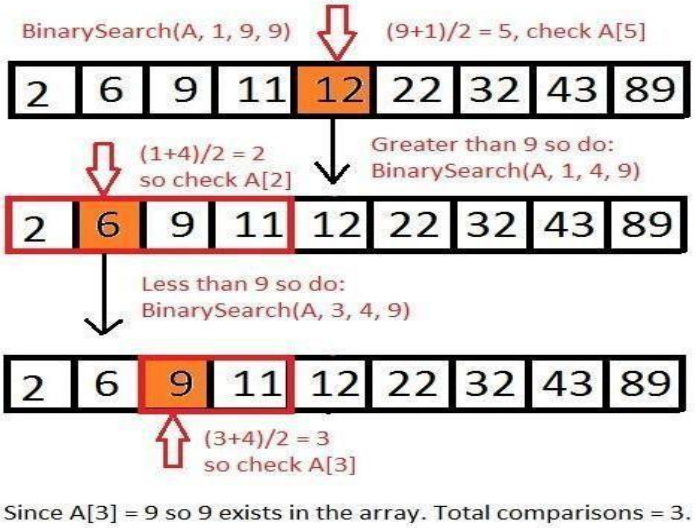
Binary search

DATA – {10, 14, 27, 28, 40, 65, 68, 89, 98, 100}
KEY- {28}

Software required: pycharm-community-2024-1.4

Theory:

A binary search or half-interval search algorithm finds the position of a specified value (the input "key") within a sorted array. In each step, the algorithm compares the input key value with the key value of the middle element of the array. If the keys match, then a matching element has been found so its index, or position, is returned. Otherwise, if the sought key is less than the middle element's key, then the algorithm repeats its action on the sub-array to the left of the middle element or, if the input key is greater, on the sub-array to the right. If the remaining array to be searched is reduced to zero, then the key cannot be found in the array and a special "Not found" indication is returned. A binary search halves the number of items to check with each iteration, so locating an item (or determining its absence) takes logarithmic time. A binary search is a divide and conquer search algorithm.



Complexity Analysis of Binary Search:

Best Case: Best case occurs when the key is in the middle of the array. Runtime will require exactly one loop.

Worst Case: Key is not in the array. It works like reducing the size of array to 1 by halving the array let m times.

$n, n/2, n/2^2 \quad n/2^m$

$n/2^m =$

1 $n=2^m$

$\log_2 n=m$

So, worst case complexity of binary search is log2n. Only one drawback of binary search affects most of the time, which is, the array must be in sorted order

Algorithms: -

Binary Search (DATA, LB, UB, ITEM, LOC):

Here DATA is a sorted array with lower bound LB and upper bound UB and ITEM is a given item of information. The variables BEG, END and MID denote, respectively, the beginning, end and middle locations of a segment of elements of DATA. This algorithm finds the location LOC of ITEM in DATA or sets LOC=NULL.

1. [Initialize segment variables.]
   Set BEG: = LB, END: =UB and MID=INT ((BEG+END)/2).
2. Repeat Steps 3 and 4 while BEG<=END AND DATA [MID]! =ITEM.
3. If ITEM< DATA[MID], then:
   Set END: =MID-1.
   Else:
   Set BEG: = MID+1.
   [End of If statement.]
4. Set MID: = INT ((BEG+END)/2).
   [End of Step 2 loop].
5. If DATA[MID] = ITEM, then:
   Set LOC: = MID.
   Else:
   Set LOC: =NULL.
   [End of If structure.]
6. Exit.

Result and Discussion:

Conclusion:

Questions:

1. What is the advantage of Binary Search over Linear? Why?
2. How to calculate MID?
3. What is the necessary condition for Binary Search?

Practical No-05

Aim: Define the stack structure and execute the push, pop operation on it.

Objective: Construct a stack of integers and perform the following Operations on it.

a. Push
b. Pop
c. Display

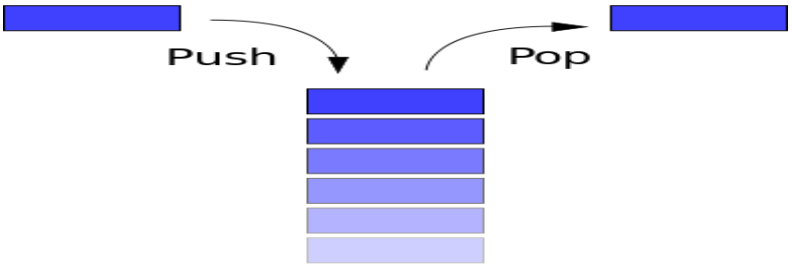
The program should print appropriate messages for stack overflow, stack underflow and stack empty.

Software required: pycharm-community-2024-1.4

Theory:

Stacks

A stack is a basic data structure, where insertion and deletion of items takes place at one end called top of the stack. This structure is used all throughout programming. The basic concept can be illustrated by thinking of your data as a stack of plates or books where you can only take the top item off the stack in order to remove things from it.A stack is also called a LIFO (Last In First Out) to demonstrate the way it accesses data. There are basically three operations that can be performed on stacks. They are 1) inserting an item into a stack (push).
2) Deleting an item from the stack (pop). 3) Displaying the contents of the stack (pip).



1. **Application of stack:-**
   a. Stacks are used to manage methods at runtime (when a method is called, its parameters and local variables are pushed onto a stack; when the method returns, the values are popped from the stack).
   b. Many parsing algorithms (used by compilers to determine whether a program is syntactically correct) involve the use of stacks.
   c. Stacks can be used to evaluate arithmetic expressions (e.g., by a simple calculator program), and they are also useful for some operations on **many** data structures.

2. **Algorithms:-**

Push (STACK,ITEM,TOP):

Description: Here STACK is an array with MAX locations. TOP points to the top most element and ITEM is the value to be inserted.

1. If (TOP == MAX) Then [Check for overflow]
2. Print: Overflow

3. Else
4. Set TOP = TOP + 1 [Increment TOP by 1]
5. Set STACK[TOP] = ITEM [Assign ITEM to top of STACK]
6. Print: ITEM inserted
[End of If]
Exit

Pop (STACK,ITEM,TOP):

Description: Here STACK is an array with MAX locations. TOP points to the top most element.

1. If (TOP == 0) Then [Check for underflow]
2. Print: Underflow
3. Else
4. Set ITEM = STACK[TOP] [Assign top of STACK to ITEM]
5. Set TOP = TOP - 1 [Decrement TOP by 1]
6. Print: ITEM deleted

[End of If]

7. Exit.

Program:

Result and Discussion:

Conclusion:

Questions:

1. What is stack?
2. Give real time example of stack?
3. What is meant by push and pop?
4. When overflow will occur in stack?

Practical No-06

Aim: Summarize the Queue structure and execute the insertion, deletion operation on it.

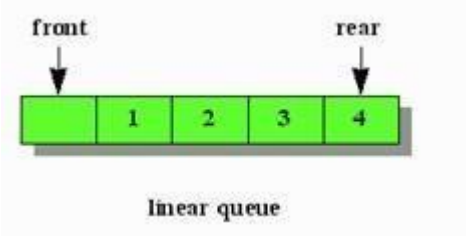
Objective: Simulate the working of a linear queue of integers using an array. Perform the following operations.

a. Insert
b. Delete
c. Display

Software Required: pycharm-community-2024-1.4

Theory: A queue is an ordered collection of items where the addition of new items happens at one end, called the "rear," and the removal of existing items occurs at the other end, commonly called the "front." As an element enters the queue it starts at the rear and makes its way toward the front, waiting until that time when it is the next element to be removed. The most recently added item in the queue must wait at the end of the collection. The item that has been in the collection the longest is at the front. This ordering principle is sometimes called FIFO, first-in first-out. It is also known as "first-come first-served."

Linear Queue: - A linear queue is like a straight line in which all elements or instructions stand one behind the other. There is a definite beginning and a definite end of the queue. Tasks lined up in this queue format are executed in the order of their placement, on a FIFO (First In First Out) basis. In a linear queue, a new task is inserted at the end of the list, while a deletion is made at the front of the list. Ex: A line for movie tickets and customers waiting at a gas station for their turn are examples of a linear queue structure.



A queue is structured, as described above, as an ordered collection of items which are added at one end, called the "rear," and removed from the other end, called the "front." Queues maintain a FIFO ordering property. The queue operations are given below.

Queue() creates a new queue that is empty. It needs no parameters and returns an empty queue.

enqueue(item) adds a new item to the rear of the queue. It needs the item and returns nothing.

dequeue() removes the front item from the queue. It needs no parameters and returns the item.The queue is modified.

isEmpty() tests to see whether the queue is empty. It needs no parameters and returns aBoolean value.

Size () returns the number of items in the queue. It needs no parameters and returns an integer.

Algorithms:-

QINSERT (QUEUE, N, FRONT, REAR, ITEM)

This procedure inserts an element ITEM into the queue.

1. [Queue is already filled?]
   If FRONT=1 and Rear=N or if Front=REAR+1, then:
   Write: Overflow, and Return.
2. [Find new value of REAR.]
   If FRONT: =NULL, then: [QUEUE is initially empty]
   Set FRONT: =1 and REAR: =1.
   Else if REAR=N,
   then:
   Set REAR: =1
   Else:
   Set REAR: =REAR+1.
   [End of if structure]
3. Set QUEUE [REAR]:=ITEM. [This inserts new element.]
4. Return.

QDELETE (QUEUE, N, FRONT, REAR, ITEM)

This procedure DELETES an element from queue and assigns it to variable ITEM .

1. [Queue is already filled?]
   If FRONT=NULL, then:
   Write: UNDERFLOW, and Return.
2. Set ITEM:=QUEUE[FRONT]
3. [Find new value of REAR.]
   If FRONT: =REAR, then: [QUEUE has only one element to start.]
   Set FRONT: =NULL and REAR: =NULL.
   Else if REAR=N, then:
   Set FRONT: =1
   Else:
   Set FRONT: =FRONT+1.
   [END OF IF STRUCTURE]
4. Return.

Program:

Result and Discussion:

Conclusion:

Questions:

1) What is a linear queue?
2) What is rear and front pointer?
3) Give real time example of queue?
4) State different applications of queue.

Practical No- 7

Aim: Discuss the Insertion sorting operation and implement on given DATA.

Objective: Implement Insertion sort for a given list of integers and display results after every pass also analyze time and space complexity for the same.

DATA – { 40,10,9,20,30,50}

Software required: pycharm-community-2024-1.4

Theory:

Insertion sort is a simple sorting algorithm that builds the final sorted array (or list) one item at a time. It is much less efficient on large lists than more advanced algorithms such as quick sort, heap sort, or merge sort. Insertion sort iterates, consuming one input element each repetition, and growing a sorted output list. At each iteration, insertion sort removes one element from the input data, finds the location it belongs within the sorted list, and inserts it there. Insertion sort is stable and has quadratic asymptotic complexity *O*(n2). If the input sequence is already sorted, than the insertion sort only checks the ordering and does not perform any moves. Hence if there are only few unsorted elements, the complexity of insertion sort gets close to *O*(n).

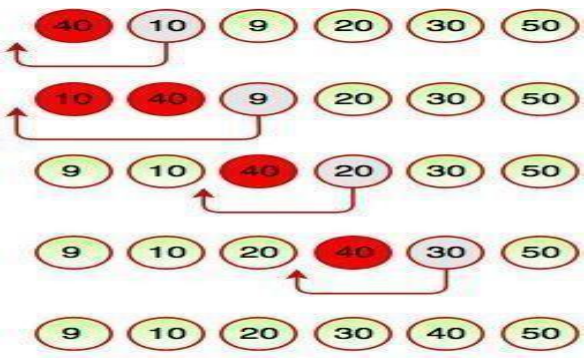


Fig. Working of Insertion Sort

Algorithm:

(Insertion Sort) **INSERTION (A,N)**

This algorithm sorts the array A with N elements-

1. Set A[0]: = −∞ [Initialize Sentinel element] 2. Repeat step 3 to 5 for k=2,3,4⋯N.
3. Set Temp: =A[k] and PTR=k-1;
4. Repeat while TEMP < A[PTR]:
    a) Set A[PTR+1]:=A[PTR]. [Move element forward]
    b) Set PTR:= PTR-1

   [End of loop]
5. Set A[PTR+1]:= TEMP [Inserts element in proper

place] [End of Step 2 loop]

6. Return

Program:

Result and Discussion:

Conclusion:

Questions:

1. What is insertion sort? Explain its sorting technique?
2. What are the different types of sorts in data structures?
3. What is the efficiency of insertion sort in worst case?
4. What is the worst case time complexity of insertion sort

Practical No-8

Aim: Implement Selection sort for a given list of integers and display results after every pass also analyze time and space complexity for the same.

DATA – { 8,15,16,17,19 }

Software required: pycharm-community-2024-1.4

Theory:

Selection sort is a simple sorting algorithm. This sorting algorithm is an in-place comparison-based algorithm in which the list is divided into two parts, the sorted part at the left end and the unsorted part at the right end. Initially, the sorted part is empty and the unsorted part is the entire list. The smallest element is selected from the unsorted array and swapped with the leftmost element, and that element becomes a part of the sorted array. This process continues moving unsorted array boundary by one element to the right. This algorithm is not suitable for large data sets as its average and worst case complexities are of $O(n^2)$, where **n** is the number of items. Selection sort is noted for its simplicity, and it has performance advantages over more complicated algorithms in certain situations, particularly where auxiliary memory is limited.

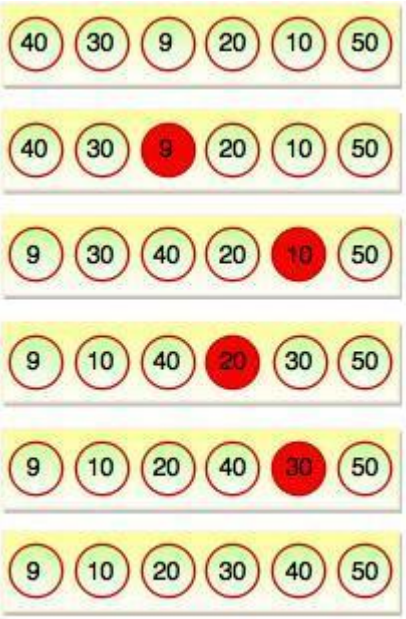


Fig. Working of Selection Sort

Algorithm:

```
Step 1 – Set MIN to location 0
Step 2 – Search the minimum element in the list
Step 3 – Swap with value at location MIN
Step 4 – Increment MIN to point to next element
Step 5 – Repeat until list is sorted
```

Program:

Result and Discussion:
Conclusion:

Questions:

1. Explain the sorting?
2. What are the different types of sorts in data structures?
3. What is Selection sort?
4. What is the efficiency of Selection sort in worst case?
5. How many passes are required in selection sort?